MyStore API Guide

Guide de reference API & Exercice React.js

json-server sur localhost:3001 244 produits • 5 categories • 19 endpoints

Sommaire

1. Ce qui vous est fourni
2. Demarrage rapide
3. Integration CSS : 3 approches possibles
4. Structure d'un produit
5. Mapping API ↔ Pages maquettes
6. API Categories
7. API Products (CRUD + Recherche)
8. API Slides (Carousel)
9. API Cart (Panier) - Cycle de vie complet
10. API Orders (Commandes)
11. Recherche full-text en detail
12. Pagination & Tri
13. Travail a realiser

1. Ce qui vous est fourni

Dossier API (Mock Server)

```
api/
├── db.json              # API complete (244 produits, 5 categories, slides, carts, orders)
├── package.json         # npm install + npm start (pas besoin d'install globale)
├── API-GUIDE.html       # Ce document de reference
└── test-api.js          # Script pour verifier que tout marche (19 tests)
```

Dossier Maquettes (Pages statiques + Assets)

```
MaquetteExo/VueJS-Exo/
├── Maquette-mini-site-exo-vueJS/       # Pages HTML statiques
│   ├── index.html                       # Page d'accueil (carousel, top sellers, top new)
│   ├── shop.html                        # Page liste produits (grille + pagination)
│   ├── product.html                     # Page detail produit
│   ├── cart.html                        # Page panier
│   ├── checkout.html                    # Page commande (formulaire billing/shipping)
│   ├── css/                             # bootstrap.min.css, font-awesome.min.css, style.c
│   ├── img/                             # Images du site (logo, slides, thumbnails)
│   ├── fonts/                           # FontAwesome + Glyphicons
│   └── js/                              # bootstrap.min.js
└── produts-img/                         # Images des produits par marque
    ├── Apple/                           # 38 images
    ├── Samsung/                         # 84 images
    ├── LG/                              # 53 images
    ├── Huawei/                          # 39 images
    └── Sony/                            # 30 images
```

> **Les maquettes HTML** sont votre reference visuelle. Ouvrez-les dans le navigateur pour voir le rendu attendu de chaque page. Votre objectif est de reproduire ces pages en React.js en remplacant les donnees statiques par des appels API.

2. Demarrage rapide

Etape 1 : Installer et lancer l'API

```
# Dans le dossier api/
npm install                 # Installe json-server localement
npm start                   # Demarre le serveur sur http://localhost:3001
```

> **Pas besoin de `npm install -g json-server` !** Le package.json inclut deja json-server en dependance locale. Un simple `npm install` + `npm start` suffit.

Etape 2 : Creer le projet React

```
# Dans un autre dossier (a cote du dossier api/)
npm create vite@latest mystore-app -- --template react
cd mystore-app
npm install
npm run dev                 # Demarre le projet React sur http://localhost:5173
```

Etape 3 : Tester l'API dans le navigateur

```
# Ouvrez ces URLs dans votre navigateur pour verifier que l'API repond :
http://localhost:3001/categories
http://localhost:3001/products?q=iphone
http://localhost:3001/products?categoryId=100&_page=1&_limit=12
http://localhost:3001/slides

# Ou lancez la suite de tests automatique :
node test-api.js            # Doit afficher 19/19 OK
```

3. Integration CSS : 3 approches possibles

Pour le style de votre application, vous avez le choix entre 3 approches. A vous de choisir celle qui vous convient le mieux :

Option A
Maquette statique

Reutilisez directement les fichiers CSS fournis dans les maquettes (bootstrap.min.css, style.css, responsive.css) et les classes HTML existantes.

```
Copiez css/ et fonts/
dans votre dossier
public/ ou src/assets/
```

Pour : Rapide, rendu identique aux maquettes
Contre : CSS legacy, pas modulaire

Moderne

Option B
SCSS / Sass

Reecrivez le CSS en SCSS avec des variables, des mixins et une architecture par composant. Inspirez-vous des maquettes pour le design.

```
npm install -D sass
// Creez vos fichiers
src/styles/
├── _variables.scss
├── _mixins.scss
├── components/
└── main.scss
```

Pour : Maintenable, reutilisable
Contre : Plus de temps

Utility-first

Option C
Tailwind CSS

Utilisez Tailwind CSS pour styler directement dans le JSX avec des classes utilitaires. Inspirez-vous des maquettes pour le design.

```
npm install -D tailwindcss
@tailwindcss/vite
npx tailwindcss init
```

Pour : Rapide, pas de fichiers CSS
Contre : Classes longues dans le JSX

> **Conseil :** Quelle que soit l'approche choisie, ouvrez les maquettes HTML dans votre navigateur pour voir le rendu attendu. Les maquettes sont votre **reference visuelle**, pas votre source de code.

4. Structure d'un produit

Chaque produit retourne par l'API contient les champs suivants :

```
{
  "id": 1015,                          // Identifiant unique
  "name": "apple iphone 11 pro max",    // Nom du produit
  "imageName": "apple-iphone-11-pro-max-.jpg",  // Nom du fichier image
  "price": 308,                         // Prix en euros
  "discountRate": 22,                   // Remise en %
  "review": 4,                          // Note de 1 a 5 (etoiles)
  "categoryId": "100",                  // ID de la categorie parente
  "categoryName": "Apple",              // Libelle de la categorie parente
  "type": "phone",                      // phone | tablet | watch
  "description": "Smartphone Apple...",   // Description du produit
  "inStock": true,                      // Disponibilite
  "isTopSeller": false,                 // Afficher dans "Top Sellers"
  "isNewProduct": false                 // Afficher dans "Top New"
}
```

> **categoryId + categoryName :** Ces deux champs sont presents sur chaque produit pour vous permettre de construire facilement un fil d'Ariane (breadcrumb) et de naviguer entre les categories sans appel API supplementaire.

Calcul du prix apres remise

```
Prix original :   308 €
Remise (22%) :    308 × 22 / 100 = 67.76 €
Prix final :      308 - 67.76 = 240.24 €
```

Chemin des images

Les images des produits sont organisees par marque dans `produts-img/{Brand}/` . Copiez-les dans votre projet React :

```
// Copiez produts-img/ dans public/images/products/
// Puis referencez-les comme ceci :

/images/products/{categoryName}/{imageName}

// Exemple : /images/products/Apple/apple-iphone-11-pro-max-.jpg
```

5. Mapping API ↔ Pages maquettes

Voici quel endpoint utiliser pour alimenter chaque page :

Page Accueil

`Maquette : index.html`

- `GET` `/categories` → Menu navigation
- `GET` `/slides` → Carousel / Slider
- `GET` `/products?isTopSeller=true` → Section "Top Sellers"
- `GET` `/products?isNewProduct=true` → Section "Top New"

Page Shop (liste produits)

`Maquette : shop.html`

- `GET` `/products?categoryId=xxx` → Grille produits
- `GET` `...&_page=1&_limit=12` → Pagination
- `GET` `...&_sort=price&_order=asc` → Tri

Page Detail Produit

`Maquette : product.html`

- `GET` `/products/:id` → Toutes les infos du produit
- Champs `categoryId` + `categoryName` → Fil d'Ariane

Page Panier

`Maquette : cart.html`

- `GET` `/carts/:id` → Contenu du panier
- `PUT` `/carts/:id` → Modifier quantites
- `DELETE` `/carts/:id` → Vider le panier

Page Checkout

`Maquette : checkout.html`

- `GET` `/carts/:id` → Recap commande
- `POST` `/orders` → Creer la commande
- `DELETE` `/carts/:id` → Vider apres commande

Barre de Recherche

`Presente sur toutes les pages`

- `GET` `/products?q=mot_cle` → Recherche globale
- `GET` `...?categoryId=xxx&q=mot` → Recherche dans une categorie

6. API Categories

Les categories representent les marques du menu de navigation.

GET `/categories` — Liste toutes les categories

Utilisation : Menu de navigation (Navbar), Footer, filtres lateraux

```
// Reponse
[
  { "id": "100",  "name": "Apple",   "image": "apple-logo.png" },
  { "id": "c200", "name": "Samsung", "image": "samsung-logo.png" },
  { "id": "c300", "name": "LG",      "image": "lg-logo.png" },
  { "id": "c400", "name": "Huawei",  "image": "huawei-logo.png" },
  { "id": "c500", "name": "Sony",    "image": "sony-logo.png" }
]
```

GET `/categories/:id` — Une categorie par ID

Exemple : `GET /categories/100` → `{ "id": "100", "name": "Apple", "image": "apple-logo.png" }`

7. API Products

L'endpoint principal du site. Supporte filtrage, recherche, pagination et tri.

GET `/products` — Tous les produits (244)

Attention : Retourne les 244 produits d'un coup. Privilegiez toujours les filtres et la pagination.

GET `/products/:id` — Detail d'un produit

Utilisation : Page detail produit

Exemple : `GET /products/1015` retourne le produit complet avec `categoryId`, `categoryName`, `description`, etc.

GET `/products?categoryId=xxx` — Produits par categorie

Utilisation : Page Shop quand on clique sur une categorie dans le menu

URL	Resultat
`/products?categoryId=100`	38 produits Apple
`/products?categoryId=c200`	84 produits Samsung
`/products?categoryId=c300`	53 produits LG
`/products?categoryId=c400`	39 produits Huawei
`/products?categoryId=c500`	30 produits Sony

GET `/products?type=xxx` Produits par type

URL	Resultat
`/products?type=phone`	218 smartphones
`/products?type=tablet`	21 tablettes
`/products?type=watch`	5 montres

Combinaison : `/products?categoryId=c200&type=tablet` → Tablettes Samsung uniquement

GET `/products?q=mot_cle` Recherche full-text

Utilisation : Barre de recherche du site

URL	Resultat
`/products?q=iphone`	21 resultats
`/products?q=galaxy`	82 resultats
`/products?q=xperia`	29 resultats
`/products?q=pro`	19 resultats (Apple + Samsung + Huawei)

Important : Le parametre s'appelle `q` (pas `search`, pas `keyword`). C'est la convention json-server pour le full-text search.

GET `/products?isTopSeller=true` Produits les plus vendus

Utilisation : Section "Top Sellers" sur la page d'accueil. Retourne 3 produits.

GET	/products?isNewProduct=true	Nouveautes

Utilisation : Section "Top New" sur la page d'accueil. Retourne 3 produits.

8. API Slides (Carousel)

GET `/slides` Slides du carousel

Utilisation : Carousel / Slider sur la page d'accueil

```
[
  { "id": 1, "image": "h4-slide.png",  "title": "New Collection 2024",  "subtitle": "
  { "id": 2, "image": "h4-slide2.png", "title": "Best Deals",           "subtitle": "
  { "id": 3, "image": "h4-slide3.png", "title": "Premium Brands",       "subtitle": "
  { "id": 4, "image": "h4-slide4.png", "title": "Free Shipping",        "subtitle": "
]
```

Les images des slides sont dans `MaquetteExo/.../img/h4-slide*.png`

9. API Cart (Panier) - Cycle de vie complet

Concept cle : json-server n'a aucune notion de session. C'est a **vous** de gerer l'identifiant du panier. Le principe : faire un `POST /carts` pour obtenir un `id`, le stocker (cookie ou localStorage), puis utiliser cet `id` pour tous les appels `GET` et `PUT` jusqu'a la confirmation de la commande.

Exigence technique : La gestion du panier (state, actions, reducers) **doit etre implementee avec Redux** (Redux Toolkit recommande). Les autres parties du site (categories, produits, recherche, slides) peuvent utiliser des `useState` / `useEffect` classiques ou tout autre approche de votre choix.

Cycle de vie du panier

1. **POST /carts → Creer le panier**
   Au premier "Add to cart", envoyer un POST pour obtenir un `id` genere par json-server.

2. **Stocker l'id (localStorage ou cookie)**
   Sauvegarder le `cartId` recu pour l'utiliser dans toutes les requetes suivantes. Le mettre aussi dans un state React.

3. **GET /carts/:id → Lire le panier**
   A chaque chargement de page ou refresh, recuperer le panier avec l'id stocke.

4. **PUT /carts/:id → Modifier le panier**
   A chaque ajout/suppression de produit ou changement de quantite, envoyer le panier complet mis a jour.

5. **POST /orders + DELETE /carts/:id → Confirmer la commande**
   Creer la commande, supprimer le panier cote serveur, puis
   supprimer le cartId du localStorage
   et reinitialiser le state. Un nouveau POST sera fait au prochain achat.

Detail des endpoints

POST `/carts` Creer un nouveau panier

Quand : Premier ajout au panier (quand aucun `cartId` n'existe dans le storage)

```
// Envoyez le panier avec le premier produit
// Headers: Content-Type: application/json
// Body :
{
  "total": 240.24,
  "subTotal": 200.20,
  "tax": 40.04,
  "items": [
    {
      "id": 1015,
      "name": "apple iphone 11 pro max",
      "imageName": "apple-iphone-11-pro-max-.jpg",
      "price": 308,
      "qty": 1
    }
  ]
}

// Reponse (json-server genere automatiquement un id) :
{
  "total": 240.24,
  "subTotal": 200.20,
  "tax": 40.04,
  "items": [...],
  "id": "xMrG3rM"         // ← STOCKER CET ID !
}
```

GET `/carts/:id` Recuperer le panier

Quand : Au chargement de l'app, si un `cartId` existe dans le localStorage, recuperer le panier pour l'afficher.

Exemple : `GET /carts/xMrG3rM`

PUT `/carts/:id` Mettre a jour le panier

Quand : A chaque modification (ajout produit, +/- quantite, suppression d'un item)

Envoyer le panier **complet** (pas juste le changement). json-server remplace tout l'objet.

Exemple : `PUT /carts/xMrG3rM` avec le body complet du panier mis a jour.

DELETE `/carts/:id` Supprimer le panier

Quand : Apres la confirmation de la commande

Exemple : `DELETE /carts/xMrG3rM`

Apres cette requete, **supprimer le cartId du localStorage** et reinitialiser le state du panier a vide.

> **Rappel :** json-server ne fait **aucune validation**. Si vous envoyez un body vide `{}`, il creera quand meme un panier avec juste un `id`. C'est a vous de valider les donnees cote front avant d'envoyer les requetes.

10. API Orders (Commandes)

POST `/orders` Creer une commande

Quand : L'utilisateur clique sur "Place order" dans le checkout

```
// Body de la requete POST /orders
{
  "total": 456.80,
  "subTotal": 380.67,
  "tax": 76.13,
  "items": [
    { "id": 1015, "name": "apple iphone 11 pro max", "price": 308, "qty": 1 }
  ],
  "customer": {
    "email": "client@email.com",
    "phone": "0612345678",
    "note": "",
    "billingAdress": {
      "civility": "Mr",
      "firstName": "Jean",
      "lastName": "Dupont",
      "zipCode": "75001",
      "street": "1 rue de Rivoli",
      "companyName": "ACME",
      "county": "France",
      "city": "Paris"
    },
    "shippingAdress": {
      // Meme structure que billingAdress
      // (si "Ship to different address" est coche)
    }
  },
  "paymentMethod": "paypal"     // "bacs" | "cheque" | "paypal"
}
```

> **Apres le POST /orders reussi :** Pensez a supprimer le panier (`DELETE /carts/:id`), vider le localStorage (`cartId`), et reinitialiser le state. Un nouveau `cartId` sera genere au prochain achat.

GET `/orders` Liste des commandes

Utilisation : Historique des commandes (bonus)

11. Recherche full-text en detail

Comment fonctionne `q=`

json-server cherche le mot dans **tous les champs texte** de chaque produit (name, description, categoryName, type). La recherche est **insensible a la casse**.

Scenario	URL	Resultats
Recherche globale	`/products?q=iphone`	21 resultats (tous les iPhone, toutes categories)
Recherche dans une categorie	`/products?categoryId=100&q=ipad`	14 resultats (iPad Apple uniquement)
Recherche + type	`/products?type=tablet&q=samsung`	Tablettes Samsung
Recherche + pagination	`/products?q=galaxy&_page=1&_limit=12`	12 resultats + header X-Total-Count: 82
Recherche + tri	`/products?q=xperia&_sort=price&_order=asc`	29 resultats tries par prix croissant
Recherche cross-categories	`/products?q=pro`	19 resultats (Apple + Samsung + Huawei)

12. Pagination & Tri

Pagination

json-server supporte la pagination nativement avec `_page` et `_limit` .

`GET /products?_page=1&_limit=12` Page 1, 12 produits par page

Header de reponse important :

```
X-Total-Count: 244          // Nombre total de produits (dans le header HTTP)
Link: <...?_page=2&_limit=12>; rel="next",  // Liens de navigation
      <...?_page=21&_limit=12>; rel="last"
```

Utilisez `X-Total-Count` pour calculer le nombre total de pages.

Tri

URL	Description
`?_sort=price&_order=asc`	Prix croissant (moins cher d'abord)
`?_sort=price&_order=desc`	Prix decroissant (plus cher d'abord)
`?_sort=review&_order=desc`	Meilleures notes d'abord
`?_sort=name&_order=asc`	Ordre alphabetique
`?_sort=discountRate&_order=desc`	Plus grosses remises d'abord

Combinaison complete

```
// Produits Samsung, type phone, page 2, 12/page, tries par prix croissant
GET /products?categoryId=c200&type=phone&_page=2&_limit=12&_sort=price&_order=asc

// Recherche "pro" dans toutes les categories, page 1, tries par note
GET /products?q=pro&_page=1&_limit=12&_sort=review&_order=desc
```

13. Travail a realiser

> **Avant de coder :** Vous devez d'abord analyser le projet, decouper le travail en taches et estimer la charge necessaire pour chaque partie. Ce travail de preparation est **obligatoire** et sera evalue.

Etape 1 : Analyse & Decoupage du projet

A partir de ce guide API et des maquettes HTML fournies, vous devez produire :

Livrable	Description
Arborescence des composants	Identifiez et listez tous les composants React necessaires a partir des maquettes (Header, Navbar, ProductCard, CartItem, CheckoutForm, etc.). Preciser pour chaque composant : son role, ses props, et dans quelle(s) page(s) il est utilise.
Liste des pages / routes	Listez toutes les pages avec leur route React Router et les endpoints API associes (cf. section 5 - Mapping).
Estimation de charge	Pour chaque section ci-dessous, estimez le temps necessaire (en heures ou en jours). Justifiez vos estimations.
Choix technique CSS	Indiquez votre choix d'approche CSS (maquette statique / SCSS / Tailwind) et justifiez-le.

Etape 2 : Sections a estimer et developper

Decoupez votre charge de travail selon les sections suivantes :

A Setup & Structure — Estimation : ? jours

Initialisation du projet, configuration de l'environnement de developpement, mise en place du routing et de l'architecture de base.

A vous d'identifier les outils, la structure de dossiers et les configurations necessaires.

B Page Accueil — Estimation : ? jours

Reproduire la page d'accueil de la maquette `index.html` en React. Referez-vous a la section 5 (Mapping) pour les endpoints a utiliser.

A vous d'analyser la maquette, d'identifier les composants necessaires et de proposer votre decoupage.

C Page Shop & Recherche

Estimation : ? jours

Reproduire la page liste de produits de la maquette `shop.html` en React, avec filtrage par categorie, pagination, tri et recherche. Referez-vous aux sections 5, 11 et 12 pour les endpoints et parametres disponibles.

A vous d'identifier les composants necessaires et de proposer votre decoupage.

D Page Detail Produit

Estimation : ? jours

Reproduire la page detail produit de la maquette `product.html` en React. Referez-vous a la section 5 (Mapping) pour les endpoints a utiliser.

A vous d'analyser la maquette, d'identifier les composants necessaires et de proposer votre decoupage.

E Panier & Checkout

Estimation : ? jours

> **Obligatoire : Cette section doit etre implementee avec Redux (Redux Toolkit recommande).** Les autres sections (A, B, C, D) peuvent utiliser `useState` / `useEffect` ou toute autre approche de votre choix.

Reproduire les pages panier (`cart.html`) et checkout (`checkout.html`) en React avec gestion d'etat Redux. Implementez le cycle de vie complet du panier decrit en section 9 et le processus de commande decrit en section 10.

A vous d'analyser les maquettes, d'identifier les composants necessaires, de proposer votre decoupage et de concevoir la structure du store Redux (slices, actions, reducers).

Etape 3 : Format du livrable de preparation

Avant de commencer le developpement, presentez un document (Markdown, Word, ou autre) contenant :

#	Section	Composants identifies	APIs utilisees	Charge estimee
A	Setup & Structure	*A vous de definir*	-	? jours
B	Page Accueil	*A vous de proposer*	*A vous d'identifier*	? jours
C	Page Shop & Recherche	*A vous de proposer*	*A vous d'identifier*	? jours
D	Page Detail Produit	*A vous de proposer*	*A vous d'identifier*	? jours
E	**Panier & Checkout (Redux)**	*A vous de proposer*	*A vous d'identifier*	? jours

#	Section	Composants identifies	APIs utilisees	Charge estimee
			Total estime	**? jours**

> **Conseil :** Soyez realistes dans vos estimations. Un bon decoupage montre que vous avez compris la complexite du projet. N'hesitez pas a identifier les risques ou les points bloquants potentiels.